

Unidad 8

Optimizadores: Descenso del gradiente estocástico (SGD), Momentum, RMSProp y Adam.
Estrategias de regularización. Hiperparámetros del entrenamiento: tamaño del lote (batch size), épocas (epochs) y tasa de aprendizaje. Aplicaciones espaciales.

Optimizadores

Un optimizer (optimizador) es un algoritmo utilizado durante el entrenamiento de una red neuronal cuyo objetivo principal es **minimizar la función de pérdida (loss function) ajustando iterativamente los parámetros del modelo (pesos y sesgos)**.

En términos simples, el optimizer es el mecanismo que le indica al modelo *cómo* y *cuánto* debe actualizar sus pesos en cada iteración para mejorar sus predicciones.

Clasificación de optimizadores:

Optimizadores clásicos de Descenso de Gradiente:

- Gradiente Descendente por Lotes (BGD - *Batch Gradient Descent*):
- Gradiente Descendente Estocástico (SGD - *Stochastic Gradient Descent*):
- Gradiente Descendente por Minilotes (*Mini-batch* GD)

Optimizadores basados en Inercia (Momento)

- SGD con Momento (*Momentum*)
- Momentos de Nesterov (NAG - *Nesterov Accelerated Gradient*)

Optimizadores con Tasa de Aprendizaje Adaptativa

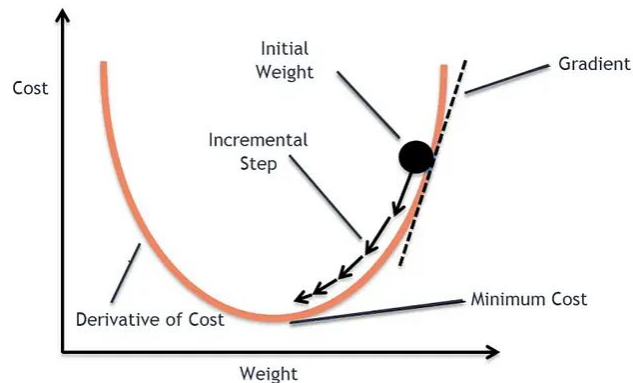
- AdaGrad
- RMSprop
- AdaDelta
- Adam (*Adaptive Moment Estimation*):
- AdamW
- Otras variantes derivadas de Adam: AdaMax, RAdam y AMSGrad
- Rprop (*Resilient Backpropagation*):
- vSGD y AdaSecant

Descenso del gradiente

Calcula el gradiente (la pendiente) de la función de pérdida con respecto a todos los pesos de la red usando todo el conjunto de entrenamiento en cada iteración.

Podemos tener

- **Gradient Descent (GD)**: Usa todas las muestras del conjunto de entrenamiento en cada paso. Es preciso pero muy lento con grandes datasets.
- **Stochastic Gradient Descent (SGD)**: Usa una sola muestra en cada paso. Es rápido y eficiente en memoria, pero muy ruidoso.
- **Mini-Batch Gradient Descent**: Usa un subconjunto (batch) de muestras en cada paso. Es el compromiso más común y práctico, equilibrando velocidad, estabilidad y eficiencia.



Gradient descent algorithm

repeat until convergence {
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(for $j = 1$ and $j = 0$)
}

Comparativa

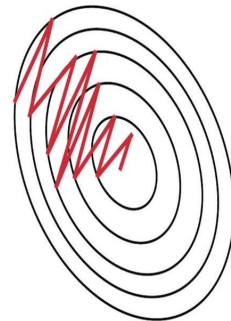
Aspect	Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
Gradient Calculation	Computes gradients using the entire dataset .	Updates the gradient using a single data point at a time.	Calculates gradients using a small subset (batch) of the dataset.
Speed	Slow due to processing the entire dataset for each update.	Fast since it updates after each data point.	Faster than batch, slower than SGD.
Memory Requirement	Requires enough memory to hold the entire dataset.	Requires minimal memory as it processes one data at a time.	Moderate memory requirements, enough for a mini-batch.
Convergence Behaviour	Converges smoothly, guarantee to reach the global minimum for convex surfaces, but may get stuck in local minima for non-convex problems.	Noisy convergence with a zigzag path, but helps escape local minima due to randomness.	Balanced—less noise than SGD but still able to escape local minima better than batch.
Variance	Low variance since all data points are considered at each update.	High variance due to reliance on a single data point per update.	Moderate variance, strikes a balance between batch and stochastic approaches.
Practical Use	Rarely used for large datasets due to inefficiency.	Useful for quick updates and initial exploration, but unstable for large-scale training.	Commonly used as it balances efficiency and stability, making it suitable for most tasks.
Example	Processes all 60,000 MNIST images in one update.	Updates after each individual image.	Divides 60,000 MNIST images into mini-batches (e.g., 128 images per batch) and updates after each batch.

Optimizadores basados en inercia

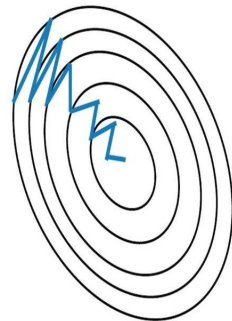
Stochastic Gradient descent (SGD) con momentum

El **Descenso de Gradiente Estocástico con Momentum** (o simplemente *SGD con momentum*) es una evolución del algoritmo de optimización SGD tradicional. Su objetivo es acelerar significativamente el proceso de entrenamiento de la red neuronal y evitar que el modelo quede atrapado en puntos subóptimos de la superficie de error.

Suaviza las oscilaciones (el efecto zigzag): Si la superficie de pérdida tiene la forma de un cañón estrecho o es rugosa, el SGD normal tiende a rebotar bruscamente de una pared a otra, ralentizando el aprendizaje. Con momentum, se calcula un promedio de los movimientos. Como resultado, las direcciones ruidosas y oscilantes (hacia los lados) se cancelan entre sí, mientras que la dirección constante y suave (hacia el fondo del valle) se suma y se refuerza.



Stochastic Gradient
Descent *without*
Momentum



Stochastic Gradient
Descent *with*
Momentum

Algorithm 1 Gradient Descent

$$\mathbf{g}_t \leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1})$$
$$\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{g}_t$$

Algorithm 2 Classical Momentum

$$\mathbf{g}_t \leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1})$$
$$\mathbf{m}_t \leftarrow \mu \mathbf{m}_{t-1} + \mathbf{g}_t$$
$$\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{m}_t$$

Como funciona matemáticamente el momentum

En lugar de actualizar los pesos usando solamente el gradiente de la iteración actual, el algoritmo mantiene un **promedio móvil ponderado exponencialmente** de los gradientes históricos. En cada paso, el algoritmo conserva una fracción de su "velocidad" anterior (generalmente controlada por un parámetro llamado β o *momentum*, cuyo valor típico suele ser 0.9) y le suma el gradiente actual. Esto significa que los gradientes antiguos van decayendo gradualmente, pero siguen influenciando la dirección actual.

Limitaciones: El principal problema del momentum es que, al acumular tanta velocidad cuesta abajo, la bola puede llegar al fondo y "**pasarse de largo**" (overshoot), subiendo por la ladera opuesta del valle antes de volver a bajar, lo que causa cierta pérdida de eficiencia al final.

Momento acelerado por Nesterov (NAG)

El **Momento de Nesterov** (o gradientes acelerados de Nesterov) es una técnica de optimización avanzada que mejora el algoritmo de Descenso de Gradiente Estocástico (SGD) con momento clásico. Fue desarrollado originalmente por Yurii Nesterov en 1983, y su adaptación al Deep Learning fue popularizada por Ilya Sutskever.

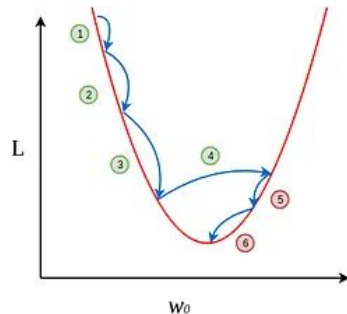
El problema que soluciona: el momento estándar actúa como una bola pesada que acumula velocidad cuesta abajo, lo que le ayuda a evadir mínimos locales. Sin embargo, tiene un defecto importante: la inercia acumulada puede ser tan grande que el algoritmo **"se pase de largo"** (*overshoot*) al llegar al punto mínimo de la función de pérdida, llevándolo a subir por la ladera opuesta antes de volver a bajar.

¿Cómo funciona (El mecanismo de "mirar hacia adelante")? Para solucionar el problema de pasarse de largo, el Momento de Nesterov dota al algoritmo de la capacidad de **"mirar hacia adelante"** (*look ahead*) en la superficie de error antes de realizar la actualización de los parámetros.

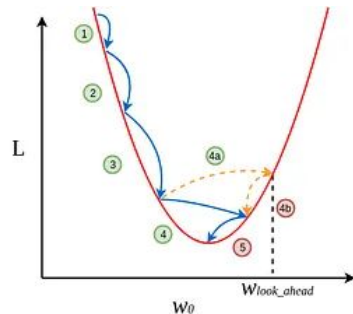
Momento acelerado por Nesterov (NAG)

Funciona en tres pasos lógicos:

1. **Salto proyectado:** En lugar de calcular el gradiente en la posición actual, el algoritmo primero realiza un salto "a ciegas" en la dirección de la velocidad que ya traía acumulada del paso anterior, llegando a un punto de destino estimado.
2. **Cálculo en el futuro:** Una vez en esa posición futura estimada, recién allí calcula el gradiente (la pendiente de la curva).
3. **Corrección:** Utiliza ese gradiente calculado en el futuro para realizar una corrección sobre el salto y determinar la actualización final de los pesos.



(a) Momentum-Based Gradient Descent



(b) Nesterov Accelerated Gradient Descent

● $\Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$

● $\Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$

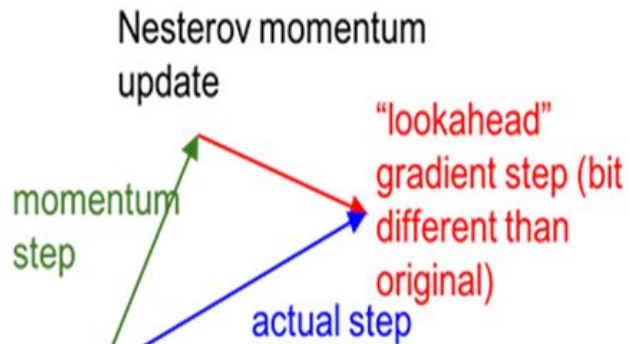
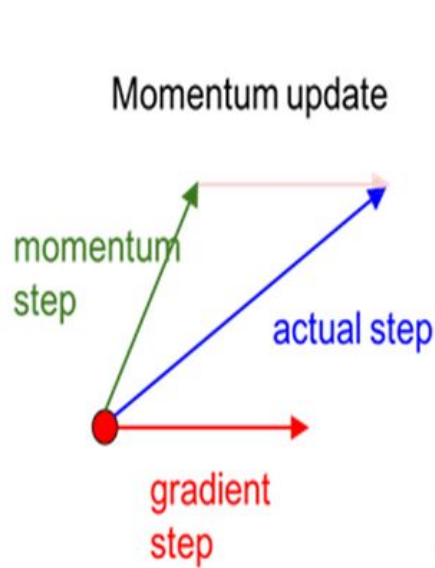
Algorithm 3 Nesterov's accelerated gradient

$$\mathbf{g}_t \leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1} - \eta \mu \mathbf{m}_{t-1})$$

$$\mathbf{m}_t \leftarrow \mu \mathbf{m}_{t-1} + \mathbf{g}_t$$

$$\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{m}_t$$

SGD con momentum vs NAG



Nesterov: the only difference...

$$v_t = \mu v_{t-1} - \epsilon \nabla f(\theta_{t-1} + \mu v_{t-1})$$

$$\theta_t = \theta_{t-1} + v_t$$

- **SGD con Momento (*Momentum*):** En lugar de usar solo el gradiente actual, acumula una media móvil de los gradientes pasados. Ayuda a acelerar la convergencia y suaviza las oscilaciones indeseadas cuando el optimizador desciende por valles irregulares.
- **Momentos de Nesterov (NAG - *Nesterov Accelerated Gradient*):** Es una mejora inteligente del momento tradicional. En lugar de evaluar el gradiente en la posición actual, estima "dónde" terminará el peso tras aplicarle el momento y evalúa el gradiente en ese punto futuro. Esto previene que el optimizador se pase de largo del mínimo

Algorithm 2 Classical Momentum

$$\mathbf{g}_t \leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1})$$

$$\mathbf{m}_t \leftarrow \mu \mathbf{m}_{t-1} + \mathbf{g}_t$$

$$\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{m}_t$$

Algorithm 3 Nesterov's accelerated gradient

$$\mathbf{g}_t \leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1} - \eta \mu \mathbf{m}_{t-1})$$

$$\mathbf{m}_t \leftarrow \mu \mathbf{m}_{t-1} + \mathbf{g}_t$$

$$\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{m}_t$$

Optimizadores con tasa de aprendizaje adaptativa

Optimizadores con tasa de aprendizaje adaptativa

AdaGrad

Adapts learning rate based on update frequency.

RMSProp

Addresses the diminishing learning rates issue.

AdaDelta

Builds upon RMSProp by limiting the window of accumulated gradients.

Adam

Combines benefits of AdaGrad and RMSProp, providing adaptive learning rates and momentum.

NAdam

Extension of Adam incorporating Nesterov momentum.

Adamax

Variant of Adam using the infinity norm for scaling.

AdaGrad (Adaptative Gradients) es un algoritmo de optimización que da un salto evolutivo respecto al Descenso de Gradiente Estocástico (SGD) tradicional. Su aporte principal es que abandona la idea de usar una única tasa de aprendizaje (learning rate) global para todos los pesos de la red y, en su lugar, introduce una **tasa de aprendizaje adaptativa individual para cada parámetro**.

¿**Cómo funciona?** Matemáticamente, AdaGrad mantiene un historial para cada parámetro del modelo, acumulando la **suma de los cuadrados de todos sus gradientes pasados**. En cada paso de actualización, el algoritmo toma la tasa de aprendizaje base o global y la divide por la raíz cuadrada de esta suma histórica acumulada (sumándole un valor minúsculo como ϵ para evitar matemáticamente dividir por cero).

1. Cálculo del gradiente $g_t = \Delta_{\theta} J(\Theta_t)$
2. Acumular gradientes cuadrados $G_t = G_{t-1} + g_t^2$
3. Actualizar los parámetros

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot g_t$$

- η is the initial learning rate,
- square root of $G_t + \epsilon$ scales the learning rate adaptively for each parameter,
- ϵ is a small constant (e.g., 10^{-8}) to prevent division by zero.

- **Ideal para datos dispersos (Sparse Data):** Gracias a esta dinámica, AdaGrad brilla especialmente en conjuntos de datos donde ciertas características (features) son muy raras. Permite que el modelo estabilice rápidamente lo que ya sabe sobre las características comunes y, al mismo tiempo, le permite "ponerse al día" rápidamente dando grandes saltos de aprendizaje cuando por fin se encuentra con las características raras.
- **Menos dependencia manual:** Al ir ajustando la tasa de aprendizaje sobre la marcha, exime al programador de tener que ajustar y reducir manualmente la tasa de aprendizaje a lo largo de las distintas épocas de entrenamiento.

El problema del decaimiento prematuro:

La mayor debilidad de AdaGrad se encuentra en el propio mecanismo de su acumulador. Dado que en cada paso de entrenamiento se suman valores siempre positivos (por estar elevados al cuadrado), **el historial acumulado en el denominador crece de forma infinita** a medida que avanzan las iteraciones.

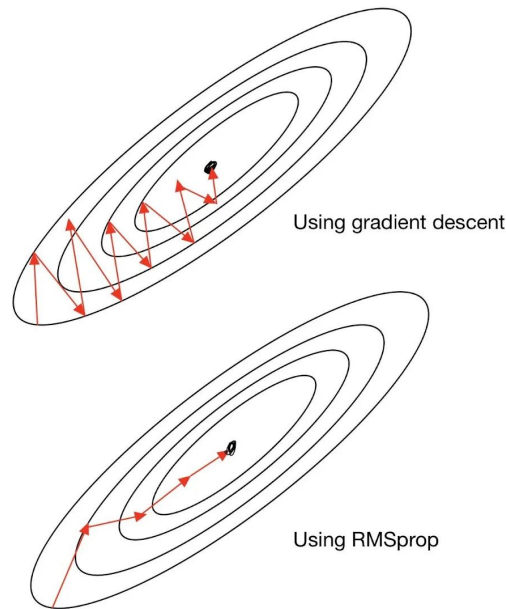
Esto provoca que la tasa de aprendizaje efectiva de todos los parámetros se vuelva cada vez más y más pequeña. Eventualmente, la tasa de aprendizaje se vuelve tan minúscula (cercana a cero) que las actualizaciones del modelo se congelan por completo; la red básicamente **deja de aprender de manera prematura y se estanca** antes de poder alcanzar el mínimo óptimo real del problema.

RMSProp (Root Mean Squared Propagation) es un algoritmo de optimización avanzado para el entrenamiento de redes neuronales, introducido originalmente por Geoffrey Hinton. Nace como una evolución directa diseñada para solucionar la mayor debilidad del algoritmo **AdaGrad** que discutimos antes: el decaimiento prematuro de la tasa de aprendizaje .

¿Cómo funciona y cómo soluciona el problema de AdaGrad?

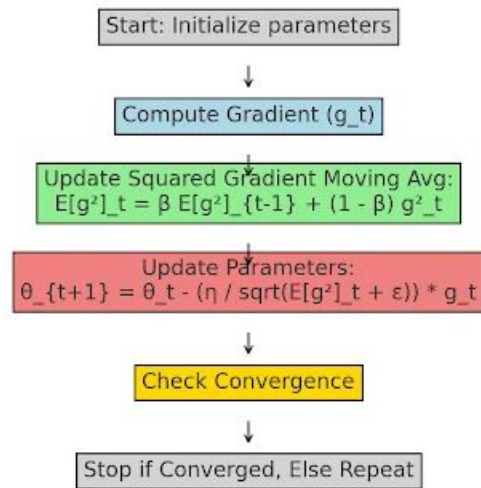
Recordemos que AdaGrad mantiene un acumulador sumando todos los gradientes al cuadrado desde el inicio del entrenamiento, lo que provoca que el denominador crezca infinitamente, la tasa de aprendizaje colapse a cero y la red deje de aprender.

RMSProp soluciona este defecto reemplazando esa suma total acumulada por una **media móvil ponderada exponencialmente** de los gradientes al cuadrado. Esto significa que, en lugar de recordar todo el historial por igual, el algoritmo "olvida" gradualmente las mediciones muy antiguas y le da más importancia a la magnitud de los gradientes recientes.



Ventajas de RMSProp

- **Aprendizaje adaptativo e ininterrumpido:** Al igual que AdaGrad, le otorga a cada parámetro de la red su propia tasa de aprendizaje personalizada. A los parámetros que cambian drásticamente les frena la velocidad para evitar que la red diverja, y a los que cambian poco les acelera el paso. La gran diferencia es que, gracias a la media móvil, la tasa de aprendizaje nunca se encoge hasta desaparecer, permitiendo que la red siga aprendiendo de forma indefinida hasta encontrar el mínimo óptimo real.
- **Suaviza el ruido en los mini-lotes (Mini-batches):** RMSProp fue concebido como una adaptación para mini-lotes del algoritmo clásico "Rprop". Su promediado móvil actúa como un amortiguador: estabiliza las fluctuaciones erráticas que ocurren cuando se procesan lotes pequeños de datos, ayudando a que la red navegue con mucha mayor fluidez a lo largo de terrenos matemáticos difíciles.



Algorithm 5 RMSProp

$$\begin{aligned} \mathbf{g}_t &\leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1}) \\ \mathbf{n}_t &\leftarrow v \mathbf{n}_{t-1} + (1 - v) \mathbf{g}_t^2 \\ \theta_t &\leftarrow \theta_{t-1} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{n}_t + \epsilon}} \end{aligned}$$

AdaDelta (Adaptive Delta) es un algoritmo de optimización propuesto por Matthew Zeiler en 2012. Nace como una extensión directa de AdaGrad con el objetivo de resolver sus dos debilidades principales: la drástica caída de la tasa de aprendizaje y la necesidad de configurar manualmente una tasa de aprendizaje inicial.

Podemos entender AdaDelta evaluando cómo soluciona estos dos problemas:

1. Solución al decaimiento de la tasa de aprendizaje (Similitud con RMSProp): Al igual que RMSProp (que se desarrolló casi al mismo tiempo), AdaDelta abandona la idea de sumar todos los gradientes al cuadrado desde el inicio del entrenamiento. En su lugar, utiliza una **media móvil ponderada exponencialmente** para acumular solo los gradientes recientes. Esto evita que el acumulador crezca hacia el infinito y asegura que la red pueda seguir aprendiendo de forma indefinida.

2. Eliminación de la Tasa de Aprendizaje global (La gran innovación): Aunque RMSProp y AdaGrad adaptan la tasa para cada parámetro, ambos siguen requiriendo que el programador elija una "tasa de aprendizaje base" (por ejemplo, $\eta=0.01$). AdaDelta va un paso más allá y **elimina por completo la necesidad de establecer una tasa de aprendizaje manual**.

$$\Delta x_t = - \frac{\eta}{\sqrt{\sum_{\tau=1}^t g_{\tau}^2}} g_t$$

Para deshacerse de la tasa de aprendizaje, el autor de AdaDelta notó que las "unidades" matemáticas en las actualizaciones de AdaGrad no coincidían correctamente con las unidades de los parámetros. Para solucionarlo (y aproximar el comportamiento de los métodos avanzados de segundo orden basados en la matriz Hessiana), AdaDelta introduce un segundo acumulador.

El algoritmo funciona manteniendo dos historiales paralelos:

1. La media móvil de los **gradientes al cuadrado** (igual que RMSProp).
2. La media móvil de las **actualizaciones al cuadrado** (es decir, el historial de los "deltas" o cambios reales que sufrieron los pesos en los pasos anteriores. De ahí proviene el nombre *AdaDelta*).

Adam (Adaptive Moment Estimation o Estimación de Momento Adaptativo) es un algoritmo de optimización avanzado que combina las mejores características del Descenso de Gradiente con **Momentum** y de **RMSProp**. Hoy en día, es considerado el optimizador estándar y la opción por defecto en la gran mayoría de las aplicaciones de aprendizaje profundo moderno debido a su rapidez y robustez.

Características principales de Adam:

- Combina ideas tanto de momentum como de RMSprop
- Mantiene una media exponencialmente decreciente de los gradientes pasados y de los gradientes pasados al cuadrado
- Índices de aprendizaje adaptativos para cada parámetro
- Corrección del sesgo de los pasos iniciales

Adam mantiene dos vectores de estado matemáticos (o acumuladores) operando en paralelo para cada parámetro de la red:

1. **El primer momento (Inspirado en Momentum):** El algoritmo calcula una media móvil ponderada exponencialmente de los *gradientes pasados*. Al igual que en el momentum clásico, esto actúa como una "velocidad" que suaviza la trayectoria y permite que la red acumule inercia en las direcciones donde el gradiente es consistente.
2. **El segundo momento (Inspirado en RMSProp):** Simultáneamente, calcula una media móvil ponderada exponencialmente de los *cuadrados de los gradientes pasados*. Al dividir la actualización por la raíz cuadrada de este segundo acumulador, Adam le otorga a cada parámetro su propia **tasa de aprendizaje adaptativa** (reduciendo el paso de los parámetros con gradientes grandes y acelerando el de los gradientes pequeños)

Corrección de sesgo en Adam

Adam soluciona un problema crítico que tenían las versiones iniciales de estos estimadores. Como ambos acumuladores se inicializan desde cero al comenzar el entrenamiento, sus valores promedio están artificialmente "sesgados" (tirados hacia abajo, hacia el cero) durante las primeras iteraciones.

Para contrarrestar esto, Adam aplica una **corrección de sesgo**. Utiliza fórmulas matemáticas que dividen ambos promedios por un factor de decaimiento basado en el número de iteración actual (ej. $1-\beta^t$). Esto amplifica los valores al inicio del entrenamiento para compensar el inicio desde cero, asegurando que las estimaciones de dirección y magnitud sean precisas y sin sesgo desde el primer paso.

La actualización final: En cada iteración, el algoritmo actualiza los pesos de la red tomando el primer momento corregido (la dirección deseada) y dividiéndolo por la raíz cuadrada del segundo momento corregido (la magnitud de la tasa adaptativa), sumando una pequeñísima constante ϵ para evitar dividir por cero.

1. **Uso de un Momento "anticuado" (El problema de la inercia)**: Adam combina el mecanismo de RMSProp con el algoritmo de **Momentum clásico**. Como discutimos anteriormente, el problema del momento clásico es que calcula su trayectoria basándose en un vector "desactualizado" que utiliza únicamente los gradientes pasados, lo que puede llevarlo a pasarse de largo al llegar a un mínimo
2. **Incompatibilidad con la regularización tradicional (Weight Decay)**: Otra desventaja importante surge en cómo Adam interactúa con ciertas técnicas de regularización que se utilizan para evitar el sobreajuste, en particular el decaimiento de pesos o *weight decay*. Investigaciones demostraron que la forma matemática tradicional de aplicar el decaimiento de pesos no funciona correctamente cuando se emplea dentro del optimizador Adam.
3. **Rendimiento inferior en ciertas arquitecturas**: Aunque Adam es un optimizador muy versátil, no es infalible para todos los problemas. En experimentos específicos entrenando modelos de lenguaje recurrentes (como los LSTM), se observó que los algoritmos que incorporan la mecánica de RMSProp (incluyendo Adam y Nadam) en realidad empeoraron el rendimiento en comparación con el Descenso de Gradiente Estocástico (SGD) puro con momento

NAdam (Nesterov-accelerated Adaptive Moment Estimation) es un algoritmo de optimización que nace como una evolución directa de Adam.

Para entender NAdam, podemos verlo como la combinación de dos piezas:

Adam = RMSProp + Momento Clásico

NAdam = RMSProp + Momento de Nesterov.

El problema de Adam que NAdam viene a solucionar: Como vimos, Adam es excelente porque combina la tasa de aprendizaje adaptativa de RMSProp con la inercia del Momento clásico. Sin embargo, el Momento clásico tiene un defecto: calcula la dirección del siguiente paso basándose en un vector "obsoleto" de gradientes pasados, lo que puede hacer que la red acumule demasiada inercia y se pase de largo del mínimo.

Dado que matemáticamente se ha demostrado que el **Momento de Nesterov (NAG)** es superior al momento clásico porque tiene la capacidad de "mirar hacia adelante" y frenar a tiempo, el creador de NAdam (Timothy Dozat) se planteó una pregunta lógica: si vamos a usar momento junto con RMSProp, ¿por qué no usar el mejor momento disponible?

- **Dirección de mayor calidad:** En algoritmos adaptativos (donde la tasa de aprendizaje puede volverse muy grande para ciertos parámetros y pequeña para otros), reducir el ruido y asegurarse de que el modelo apunte en la dirección correcta es crítico. El Momento de Nesterov produce una actualización de dirección muy superior a la del momento clásico.
- **Mejores resultados en la práctica:** En los experimentos documentados en las fuentes (como el entrenamiento de representaciones de palabras *word2vec* o clasificación de imágenes MNIST), NAdam demostró superar a Adam de forma consistente, logrando una convergencia más rápida y menor error en las pruebas

AdaMax es una variante del algoritmo de optimización **Adam**.

Características principales:

- **Cambio a la norma L_∞ :** La diferencia central es que, mientras Adam calcula la magnitud del paso utilizando la norma L_2 (basada en promediar los gradientes elevados al cuadrado), AdaMax la sustituye por la norma L_∞ .
- **Mecanismo de valor máximo:** Matemáticamente, en lugar de sumar los cuadrados, el algoritmo actualiza su denominador eligiendo el **valor máximo** entre el historial acumulado en el paso anterior (multiplicado por un factor de decaimiento ν) y el valor absoluto del gradiente actual ($|g_t|$).
- **Mayor simplicidad (Sin corrección de sesgo):** Gracias a este enfoque basado en el valor máximo, AdaMax simplifica las operaciones y elimina la necesidad de aplicar la "corrección de sesgo" (*bias correction*) para este parámetro, un cálculo que sí es obligatorio en Adam.
- **Rendimiento en la práctica:** A pesar de ser una alternativa matemática interesante, las pruebas experimentales han demostrado que los métodos basados en esta arquitectura (como AdaMax) **generalmente obtienen peores resultados** en comparación con otros algoritmos como el propio Adam original o Nadam.

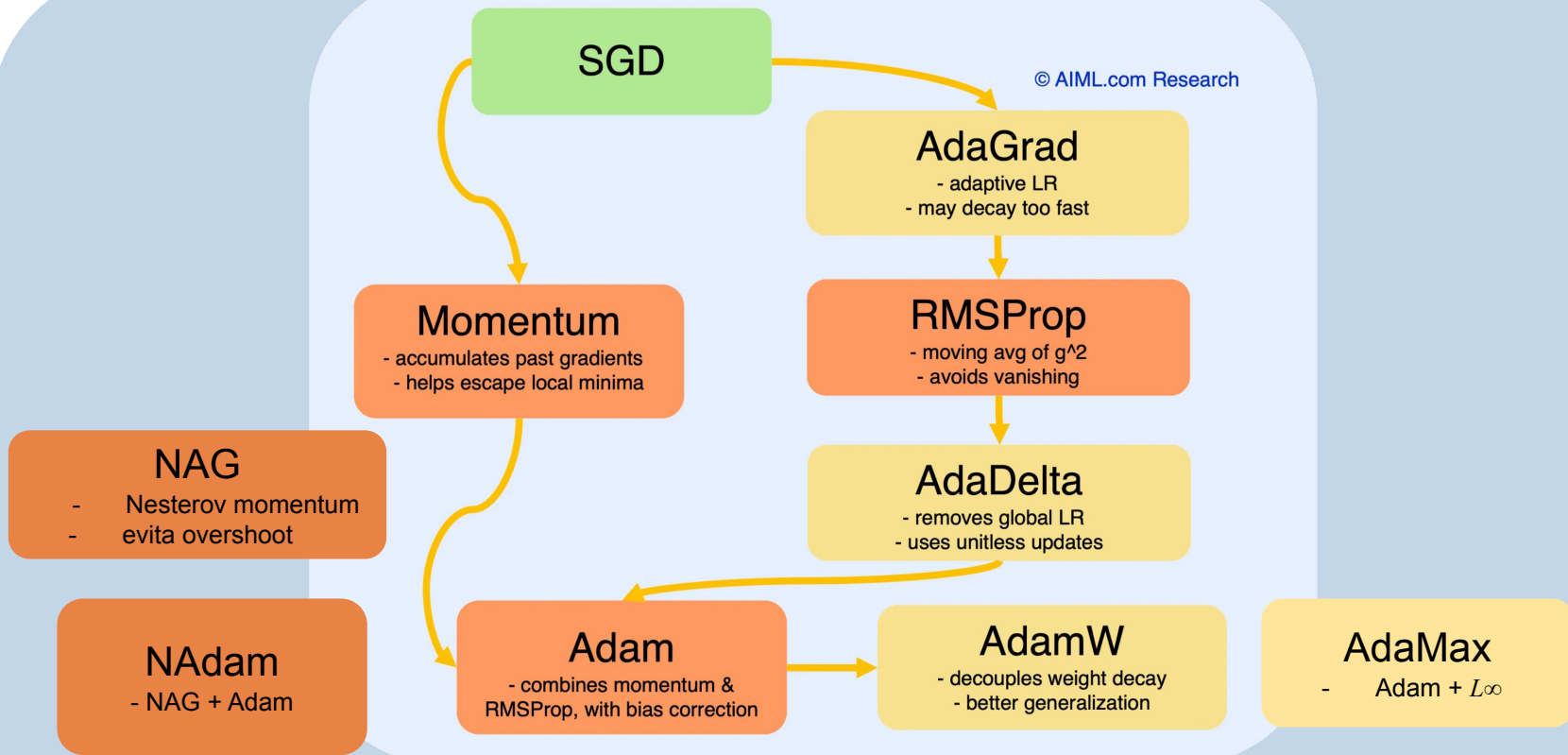
AdamW es una variante del optimizador Adam diseñada para solucionar un problema matemático fundamental en la forma en que este algoritmo aplica la regularización, específicamente la técnica de **Weight Decay** (decaimiento de pesos).

El problema de Adam con la regularización: En el algoritmo clásico de Descenso de Gradiente Estocástico (SGD), aplicar la "Regularización L2" (sumar los parámetros al cuadrado a la función de pérdida) es matemáticamente equivalente a aplicar "Weight Decay" (restar directamente una fracción del peso en cada actualización). Sin embargo, en algoritmos adaptativos como Adam, **esta equivalencia matemática se rompe por completo**. Como Adam altera el tamaño del paso de aprendizaje utilizando medias móviles e inercias de los gradientes, si incluimos la regularización dentro del cálculo del gradiente, la penalización sufre esas mismas alteraciones y pierde gran parte de su efectividad.

La solución de AdamW (El "Desacoplamiento"): Para resolver esta incompatibilidad, los investigadores Ilya Loshchilov y Frank Hutter introdujeron el concepto de **Decaimiento de Pesos Desacoplado** (*Decoupled Weight Decay*). Lo que hace AdamW es separar la regularización de la función de pérdida: el algoritmo calcula los gradientes y el tamaño del salto de forma normal (usando la mecánica tradicional de Adam) y, recién en el último paso, **"desacopla" la penalización restando el decaimiento directamente al valor final del peso**.

Evolution of Optimizers

© AIML.com Research



Estos algoritmos modernos ajustan automáticamente la tasa de aprendizaje para cada parámetro individualmente, sin necesidad de afinarla tanto de forma manual:

- **AdaGrad:** Adapta la tasa de aprendizaje disminuyéndola drásticamente para las características o parámetros que reciben actualizaciones muy grandes o frecuentes. El problema es que con el tiempo la tasa decae demasiado y la red puede dejar de aprender prematuramente.
- **RMSprop:** Creado para solucionar el problema de AdaGrad. Utiliza una media móvil exponencial de los gradientes al cuadrado, lo que evita que la tasa de aprendizaje disminuya hasta cero

Algorithm 4 AdaGrad

$$\begin{aligned}\mathbf{g}_t &\leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1}) \\ \mathbf{n}_t &\leftarrow \mathbf{n}_{t-1} + \mathbf{g}_t^2 \\ \theta_t &\leftarrow \theta_{t-1} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{n}_t + \varepsilon}}\end{aligned}$$

Algorithm 5 RMSProp

$$\begin{aligned}\mathbf{g}_t &\leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1}) \\ \mathbf{n}_t &\leftarrow v\mathbf{n}_{t-1} + (1-v)\mathbf{g}_t^2 \\ \theta_t &\leftarrow \theta_{t-1} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{n}_t + \varepsilon}}\end{aligned}$$

Resumiendo

- **AdaDelta:** Otra alternativa similar a RMSprop, diseñada para ser robusta y prescindir de una tasa de aprendizaje predeterminada.
- **Adam (Adaptive Moment Estimation):** Es el optimizador por defecto en el 90% de las tareas de *deep learning*. Combina las mejores ideas del Momento (estimando el primer momento de los gradientes) y de RMSprop (estimando el segundo momento o varianza)
- **Otras variantes derivadas de Adam:** **AdaMax** (variante que cambia la norma matemática L2 por una norma infinita), **NAdam** (incorpora los momentos de Nesterov dentro de Adam), **RAdam** y **AMSGrad** (creada para corregir casos teóricos donde Adam no converge)

Algorithm 5 RMSProp

$$\begin{aligned}\mathbf{g}_t &\leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1}) \\ \mathbf{n}_t &\leftarrow v\mathbf{n}_{t-1} + (1-v)\mathbf{g}_t^2 \\ \theta_t &\leftarrow \theta_{t-1} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{n}_t + \epsilon}}\end{aligned}$$

Algorithm 6 Adam

$$\begin{aligned}\mathbf{g}_t &\leftarrow \nabla_{\theta_{t-1}} f(\theta_{t-1}) \\ \mathbf{m}_t &\leftarrow \mu\mathbf{m}_{t-1} + (1-\mu)\mathbf{g}_t \\ \hat{\mathbf{m}}_t &\leftarrow \frac{\mathbf{m}_t}{1-\mu^t} \\ \mathbf{n}_t &\leftarrow v\mathbf{n}_{t-1} + (1-v)\mathbf{g}_t^2 \\ \hat{\mathbf{n}}_t &\leftarrow \frac{\mathbf{n}_t}{1-v^t} \\ \theta_t &\leftarrow \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{n}}_t + \epsilon}}\end{aligned}$$

Regularización

Penalizaciones en la función de coste (Regularización explícita)

- Regularización L2 (Weight Decay)
- Regularización L1 (Lasso)
- Redes elásticas (Elastic Nets)

Modificaciones estructurales y de la red

- Dropout
- Reducción del tamaño de la red
- Pesos compartidos (*Weight sharing*): en CNN

Ajustes durante el proceso de entrenamiento

- Parada Temprana (*Early Stopping*)
- Normalización por lotes (*Batch Normalization*)

Regularización L2 (Weight decay)

Regularización explícita

El **Weight Decay** (decaimiento de pesos) es una de las técnicas de regularización más fundamentales y utilizadas en el aprendizaje profundo. Su propósito principal es combatir el **sobreajuste (overfitting)**, evitando que el modelo memorice los datos de entrenamiento y mejorando su capacidad para generalizar en datos nunca antes vistos.

¿Cómo funciona? La técnica consiste en modificar la función de pérdida original del modelo añadiéndole una penalización (un costo adicional) proporcional a la suma de los cuadrados de todos los pesos de la red. Matemáticamente, por cada peso w , se añade un término a la función de error (usualmente representado como $1/2 \lambda w^2$, donde λ es el hiperparámetro que define la fuerza de la regularización).

Que aporta:

- Aplica Navaja de Ockham.
- Suaviza el modelo.
- Filtra lo irrelevante.

Regularización L1 (lasso)

Regularización explícita

La **regularización L1** es una técnica utilizada para combatir el sobreajuste (*overfitting*) en el aprendizaje profundo. Al igual que el *Weight Decay* (regularización L2), funciona añadiendo un costo adicional a la función de pérdida, pero en lugar de usar los pesos al cuadrado, la regularización L1 añade una penalización proporcional al **valor absoluto** de los coeficientes de los pesos (también conocido como la norma L1).

A nivel matemático, por cada peso w en la red neuronal, se suma un término $\lambda |w|$ a la función de error, donde λ es un hiperparámetro que determina la fuerza de dicha regularización

Sus características principales y cómo se compara con la regularización L2 son las siguientes:

- **Vectores de pesos dispersos (Sparsity):** Mientras que la regularización L2 (Weight Decay) tiende a encoger los pesos creando valores pequeños y difusos, la regularización L1 fuerza activamente a que muchos de los pesos se vuelvan **exactamente cero** durante el proceso de optimización, creando modelos "dispersos" o *sparse*
- **El motivo matemático (gradiente constante):** La razón por la que L1 produce ceros exactos radica en su derivada. En L1, el gradiente de la penalización es una constante (básicamente es 1 si el peso es positivo, o -1 si es negativo)
- **Resistencia al ruido:** Al forzar múltiples pesos a cero, las neuronas terminan "desconectando" ciertas señales y utilizando solo un pequeño subconjunto de las entradas más importantes.

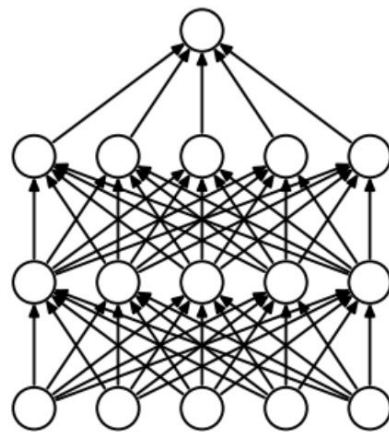
Regularización explícita

La **regularización Elastic Net** (Red Elástica) es una técnica híbrida que busca obtener lo mejor de dos mundos: **combina linealmente la regularización L1 y la regularización L2 (Weight Decay)** dentro de la misma función de pérdida.

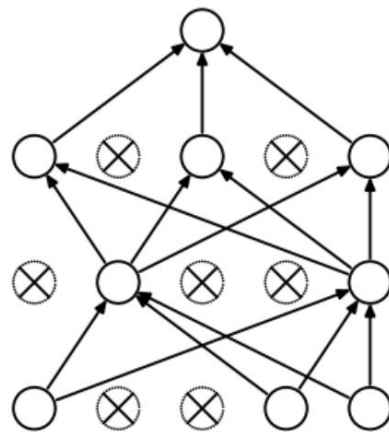
- **Combina dispersión y estabilidad:** Al heredar las propiedades de la regularización L1, Elastic Net fuerza a que varios pesos se vuelvan exactamente cero, produciendo vectores de pesos dispersos (*sparse*) que ayudan a que el modelo seleccione solo las características más importantes. Al mismo tiempo, al incluir la regularización L2, mantiene a raya el crecimiento descontrolado de los pesos restantes, fomentando que sean pequeños y dando estabilidad matemática al entrenamiento.
- **Mayor rendimiento general:** Los documentos señalan que esta combinación a menudo **funciona mejor y logra un rendimiento superior** que si se utilizara exclusivamente la regularización L1 o L2 por separado para combatir el sobreajuste (overfitting).

Modificaciones de la red

Es una de las técnicas más populares. Durante el entrenamiento, apaga (establece a cero) aleatoriamente una fracción de las neuronas (típicamente el 50%) en cada iteración. Esto evita que las neuronas se coadapten y dependan de otras unidades específicas, obligando a la red a aprender representaciones redundantes y robustas. Una variante es **DropConnect**, que apaga las conexiones (los pesos) en lugar de las neuronas enteras



(a) Standard Neural Net

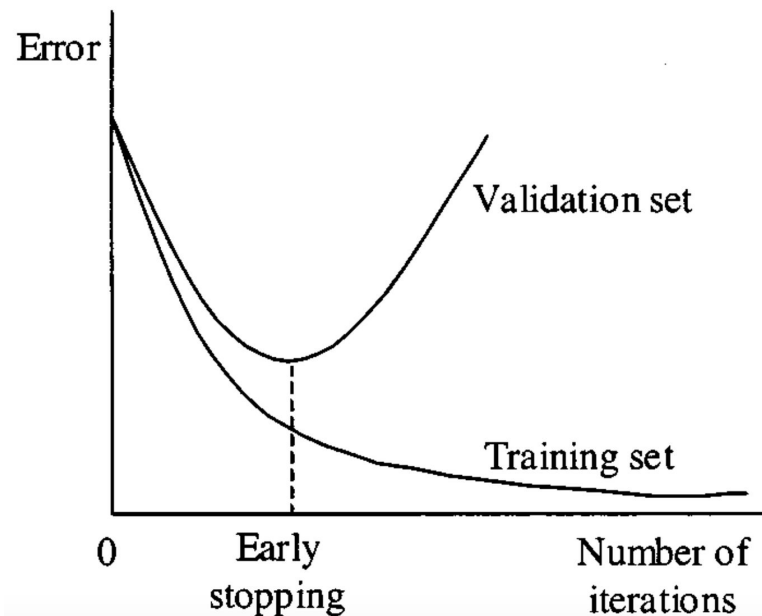


(b) After applying dropout.

Early stopping

Ajustes durante el proceso de entrenamiento

El modelo se evalúa constantemente durante el entrenamiento usando un conjunto de validación separado. El entrenamiento se detiene en el momento en el que el error de validación deja de mejorar o empieza a subir, indicando que el sobreaprendizaje ha comenzado. Se guarda el modelo de la mejor época.



Batch normalization

Ajustes durante el proceso de entrenamiento

La normalización por lotes se utiliza para reducir el problema del desplazamiento de covariables internas (internal covariance shift) en las redes neuronales. Funciona normalizando los datos dentro de cada minibatch. Esto significa que calcula la media y la varianza de los datos en un lote y luego ajusta los valores para que tengan un rango similar. Posteriormente, escala y desplaza los valores para que el modelo aprenda de forma eficaz.

La normalización por lotes garantiza que las salidas de cada capa se mantengan estables durante el aprendizaje del modelo. Esto permite un entrenamiento más rápido y un aprendizaje más eficaz.

<https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/>

Ajustes durante el proceso de entrenamiento

¿Cómo funciona matemáticamente?

Para cada capa donde se aplica, y para cada mini-lote (*mini-batch*) de datos durante el entrenamiento, la operación realiza cuatro pasos:

1. **Media del lote:** Calcula la media empírica (μ) de los valores para ese mini-lote.
2. **Varianza del lote:** Calcula la varianza (σ^2) del mini-lote.
3. **Normalización:** Centra los datos restando la media y los divide por la desviación estándar (sumando un valor muy pequeño ϵ para evitar la división por cero si la varianza es nula). Esto deja las activaciones con media cero y varianza unitaria.
4. **Escalado y desplazamiento (*Scale and shift*):** El resultado normalizado se multiplica por un parámetro γ (gamma) y se le suma un parámetro β (beta).

Ajustes durante el proceso de entrenamiento

Principales beneficios

- **Permite usar tasas de aprendizaje más altas:** Al hacer que la superficie de pérdida sea más suave y predecible, evita problemas de gradientes que explotan o desaparecen, lo que permite que la red converja mucho más rápido y de manera más estable.
- **Independencia de la inicialización:** Reduce drásticamente la sensibilidad del modelo a los valores con los que se inicializan los pesos, lo cual es vital para entrenar redes muy profundas (como las redes residuales o ResNets).
- **Efecto regularizador natural:** La normalización de un dato individual depende de las estadísticas del mini-lote específico en el que se seleccionó aleatoriamente. Esto inyecta un ruido aditivo y multiplicativo en el proceso de entrenamiento que contribuye a que el modelo generalice mejor y previene el sobreaprendizaje (*overfitting*). De hecho, en ocasiones su uso permite eliminar técnicas explícitas de regularización como el *Dropout*.

Inicializadores

En el deep learning, los **inicializadores** son los mecanismos encargados de establecer los valores de partida para los parámetros de una red neuronal (los pesos de las sinapsis y los sesgos de las neuronas) antes de que comience el proceso de entrenamiento.

Esta etapa es un factor crítico para el éxito del modelo: una mala inicialización puede hacer que el algoritmo de optimización fracase numéricamente, que el aprendizaje sea innecesariamente lento, o que el modelo se quede atascado en mínimos locales de muy mala calidad.

1. Romper la simetría (El problema de inicializar con ceros)

Un error de principiante muy habitual es inicializar todos los pesos a cero. Si dos neuronas de una misma capa reciben las mismas entradas y tienen exactamente los mismos pesos iniciales, el algoritmo de propagación hacia atrás (backpropagation) calculará el mismo gradiente para ambas. Como resultado, ambas neuronas se actualizarán idénticamente en cada iteración y aprenderán exactamente la misma característica, perdiendo toda la capacidad expresiva de tener múltiples neuronas y comportándose en la práctica como si la capa tuviera una sola neurona. Para romper esta simetría, los pesos deben inicializarse siempre de forma aleatoria.

2. El problema de la escala (Gradientes que desaparecen o explotan)

El tamaño (la escala) de los valores aleatorios iniciales tiene un impacto gigantesco:

- **Si los pesos son muy grandes:** Las activaciones crecerán exponencialmente conforme la señal avance por la red. Esto puede causar el problema de la **explosión del gradiente**, o hacer que las funciones de activación (como las sigmoideas) se saturen de inmediato, volviendo sus derivadas casi nulas y deteniendo el aprendizaje por completo.
- **Si los pesos son muy pequeños:** Las activaciones se irán reduciendo hasta cero. Durante la propagación hacia atrás, esto provoca el problema de la **desaparición del gradiente** (*vanishing gradient*), impidiendo que las primeras capas de la red se actualicen.

Principales Inicializadores de Pesos

Para solucionar los problemas de escala, se han desarrollado inicializadores matemáticamente calibrados en función del número de conexiones de entrada (*fan-in*) y salida (*fan-out*) de las neuronas:

- **Inicialización Aleatoria Escalada:** Tradicionalmente se utilizaban distribuciones (gaussianas o uniformes) con una media de cero y una varianza inversamente proporcional al número de entradas de la neurona ($1/n$). Esto evita que la suma ponderada de las entradas crezca desproporcionadamente con el tamaño de la capa.

- **Inicialización de Xavier (o Glorot):** Diseñada para intentar mantener constante la varianza de las activaciones y de los gradientes a través de todas las capas de la red. Calcula los límites de una distribución uniforme tomando en cuenta tanto las entradas (n_{in}) como las salidas (n_{out}) de la capa: $U(-\frac{n_{in}+n_{out}}{n_{in}+n_{out}}, \frac{n_{in}+n_{out}}{n_{in}+n_{out}})$.

Funciona excelentemente con **funciones de activación lineales o simétricas** como la tangente hiperbólica.

- **Inicialización de He (o Kaiming):** La inicialización de Xavier falla cuando se usan unidades ReLU (Rectified Linear Units), porque la función ReLU anula exactamente la mitad de los datos (los valores negativos). Para compensar esta pérdida de varianza, la inicialización de He la **duplica**, utilizando una varianza de $2/n_{in}$.

Típicamente se extraen los pesos de una distribución normal $N(0, 2/n_{in})$ o uniforme $U(-6/n_{in}, 6/n_{in})$.

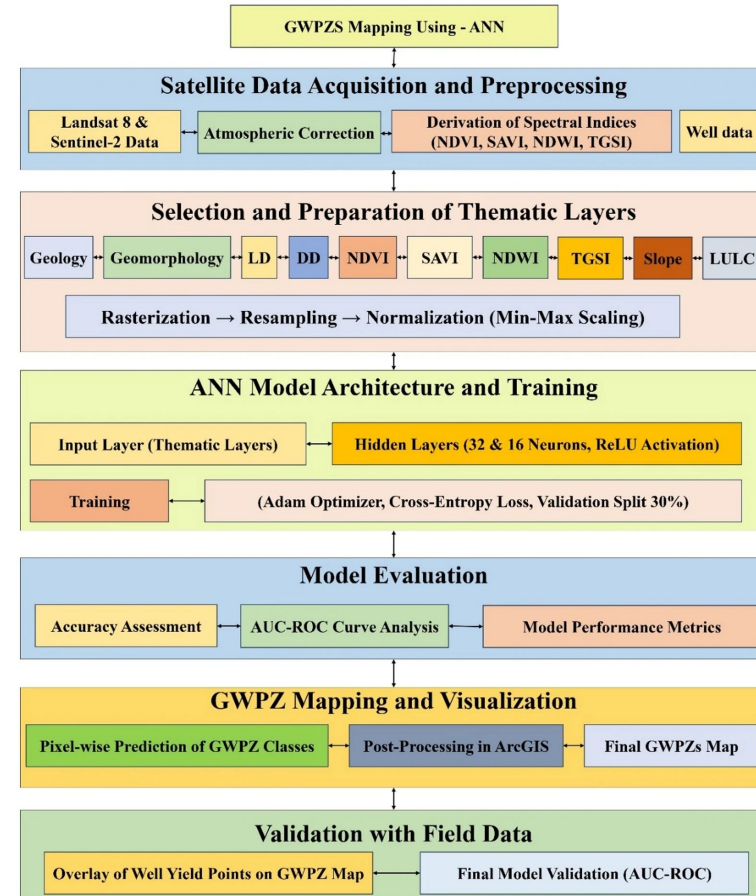
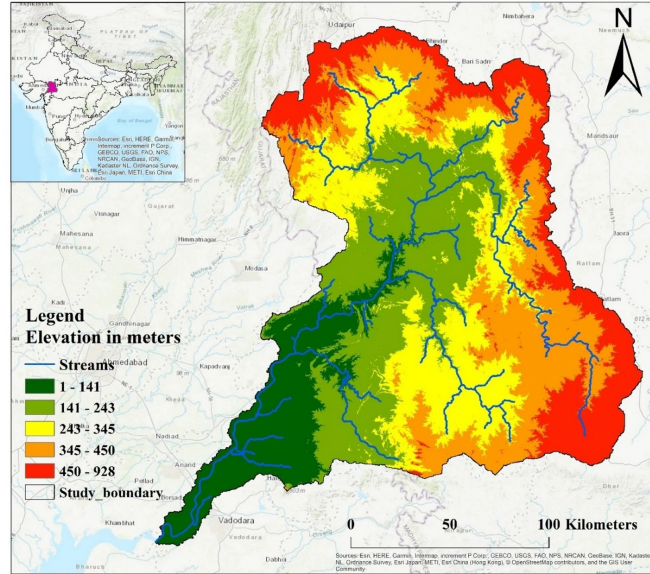
- **Inicialización Ortogonal:** Utiliza matrices ortogonales aleatorias para inicializar los pesos. Esta técnica asegura que la norma (la magnitud) de los gradientes se preserve a lo largo de la red, garantizando que el número de iteraciones requeridas para converger sea independiente de la profundidad de la red, lo que es vital para redes extremadamente profundas.
- **Inicialización Dispersa (*Sparse*):** En lugar de reducir la escala de todos los pesos cuando hay muchas conexiones, este método asigna exactamente k pesos distintos de cero a cada neurona. Crea gran diversidad en la inicialización sin que la magnitud de los pesos individuales se encoja drásticamente.

		Distribution			
		Uniform		Normal	
		Scaled by 1	Scaled by $\sqrt{6}$	Scaled by 1	Scaled by $\sqrt{2}$
Scaled by the inverse square root of:	fan-in of the layer	The heuristic prevalent when Xavier Glorot and Yoshua Bengio wrote their paper in 2010. In Keras: "uniform He initializer"		Normal Xavier (or Glorot) initialization Given in the course as "Xavier Initialization" ("Improving Deep Neural Networks", Week 1, "Weight Initialization for Deep Networks") In Keras: "normal LeCun initializer"	He initialization He, Zhang, Ren, Sun propose this in their 2015 paper as "equation 14". In Keras: "normal He initializer"
	sum of fan-in and fan-out of the layer		Uniform Xavier (or Glorot) initialization Proposed by Xavier Glorot and Yoshua Bengio in 2010. In Keras: "uniform Glorot initializer"		In Keras: "normal Glorot initializer"

Initializer	Activation Functions	Best For
Zero Initialization	None	Simple architectures, rarely used
Random Initialization	Any	Small networks
Xavier (Glorot)	Sigmoid, Tanh	Shallow to moderately deep networks
He Initialization	ReLU	Deep networks (CNNs)
Lecun Initialization	Sigmoid, Tanh	Shallow networks
Orthogonal Initialization	Any	RNNs, very deep networks
Variance Scaling	Any	Custom architectures

Aplicaciones Espaciales

Integration of geospatial techniques and artificial neural networks for groundwater potential zonation in India



AmericaView West Virginia View Courses Dark Mode Light Mode White Mode

Geospatial Deep Learning

This course is an applied study of deep learning methods for extracting information from geospatial data, such as aerial imagery, multispectral imagery, digital terrain data, and other digital cartographic representations. We have provided background and conceptual information in the form of lecture modules, video examples using ArcGIS Pro, and coding examples using Python and PyTorch or R and geodl. We primarily focus on convolutional neural networks (CNNs) and semantic segmentation methods. As we continue to expand the course, we plan to add more materials associated with transformer architectures.

The material is divided into four sections:

1. Section 1: Intro and Fully Connected Neural Networks
2. Section 2: Convolutional Neural Networks
3. Section 3: Semantic Segmentation
4. Section 4: Additional Applications

It is assumed that you have prior knowledge of coding in the Python language. The R language is also used but to a lesser extent. If you do not have experience coding, please take a look at our [Methods in Open Science](#) and/or [Open-Source Spatial Analytics \(R\)](#) courses, which explore coding in Python and/or R.

After completing this course you will be able to:

- explain how ANNs work including weights, bias, activation, and optimization.
- describe and explain different loss and assessment metrics and determine appropriate use cases.
- use the tensor data model to represent data as input for deep learning.
- explain how CNNs work including convolutional operations/layers, kernel size, stride, padding, max pooling, activation functions, and batch normalization.
- use PyTorch, Python, and R to prepare data, produce and assess scene classification

Recursos en línea

<https://medium.com/@sushmita2310/types-of-optimizers-in-deep-learning-a-comprehensive-guide-af258d6acf66>

<https://www.analyticsvidhya.com/blog/2024/11/adaptive-gradient-optimizer/>

<https://www.analyticsvidhya.com/blog/2021/06/guide-to-gradient-descent-and-its-variants-with-python-implementation/>

<https://www.datacamp.com/es/tutorial/adagrad-optimizer-explained>

<https://www.datacamp.com/es/tutorial/adam-optimizer-tutorial>